

作业 2：常量传播和 Worklist 求解器

南京大学《软件分析》课程

作业出品：谭添、李樾

1 作业导览

- 为 Java 实现常量传播算法。
- 实现一个通用的 worklist 求解器，并用它来解决一些数据流分析问题，例如本次的常量传播。

在本次实验作业中，你要在 Tai-e 框架下实现常量传播算法和 worklist 求解器的关键部分。本手册会给出算法实现的相关指引，请仔细阅读。

在阅读接下来的内容前，请先按照 **Tai-e 框架（教学版）配置指南** 将作业 2 对应的 Tai-e 项目（位于实验作业仓库的 A2/tai-e/）配置好。

2 实现常量传播

2.1 分析范围

在此次作业中，你需要实现针对 int 类型的常量传播。注意，在 Java 中，boolean、byte、char 和 short 类型在运行时实际上都以 int 值的形式进行表示和计算 (<https://docs.oracle.com/javase/specs/jvms/se11/html/jvms-2.html#jvms-2.11.1-320>)，因此你的分析算法也应当能处理这些类型。其他基本数据类型（例如 long、float 和 double）以及引用类型（例如 class types、array types）不在此次作业的考虑范围内，所以你可以在分析时忽略它们的值。

至于语句的处理，你只需要关注等号左侧为变量且右侧只能是如下几类表达式的赋值语句：

- 常量，如 $x = 1$
- 变量，如 $x = y$
- 二元运算表达式，如 $x = a + b$ 和 $x = a \gg b$

Tai-e

算法能正确处理它们：

运算类型	运算符
Arithmetic	+ - * / %
Condition	== != < > <= >=
Shift	<< >> >>>
Bitwise	& ^

逻辑运算符怎么办？

在上表中，我们没有列出逻辑运算符。这是因为，在 Java 中，逻辑运算符与（&&）和或（||）没有对应的字节码表示，而是以分支跳转的形式实现的。例如以下语句：

1

a = b || c;

java

将被转换为如下的语义等价的语句：

1

2

3

4

5

6

7

8

if (b) {
 t = true; // t is temp variable
} else if (c) {
 t = true;
} else {
 t = false;
}
a = t;

java

由于作业中的常量传播算法能够处理常量赋值（如 `t = true`）、变量赋值（如 `a = t`）和分支语句（如 `if (b) {...}`），所以它自然能够处理 `&&` 和 `||`。

至于等号左侧为变量、等号右侧为其它表达式的赋值语句，例如方法调用（`x = m(...)`）和字段 load（`x = o.f`），你需要对它们进行保守的近似处理（也许会不够精确），即把它们当作 `x = NAC`。后续的作业将逐步解锁方法调用和字段访问的精确分析技巧。

Tai-e

2.2 Tai-e 中你需要了解的类

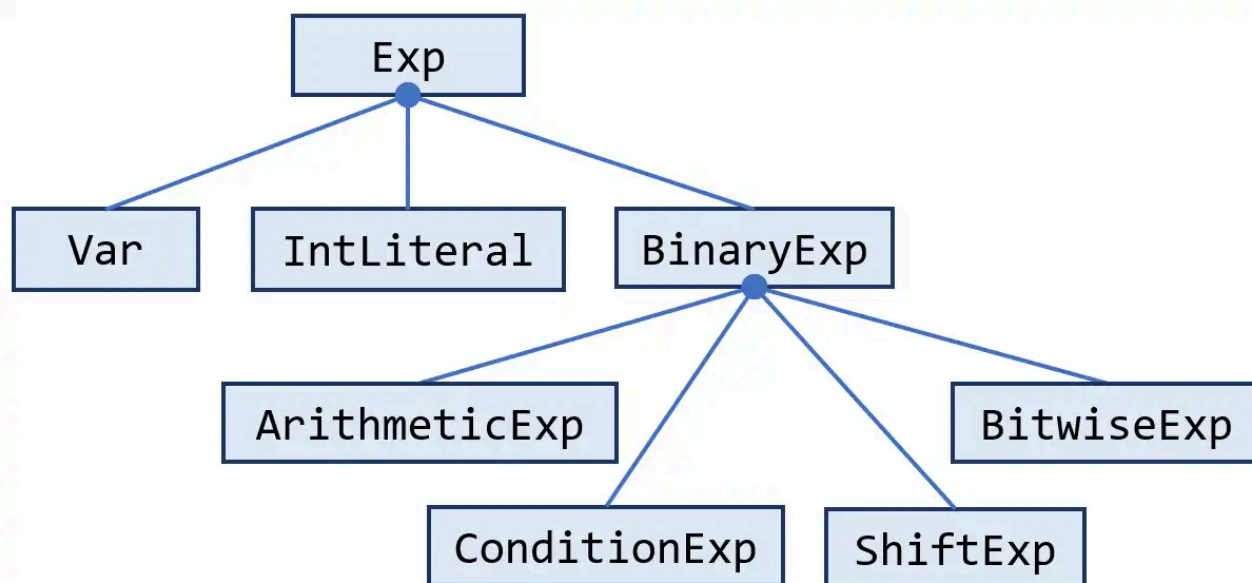
这一节将会介绍与 IR 相关的类 (`pascal.taie.ir.*`) 以及和本次的分析算法相关的类 (`pascal.taie.analysis.*`)。细致地了解它们将有助于完成常量分析算法。

- `pascal.taie.ir.IR`

这是 Tai-e 的 IR 的核心数据结构。它的每个实例储存了一个 Java 方法的各种信息，例如变量、参数、语句等等。如果想要加深理解，你可以自行阅读 API、源码实现以及注释。

- `pascal.taie.ir.exp.Exp`

我们已经在作业 1 中介绍过了这个接口。在本次作业中，你需要处理更多 `Exp` 接口的子类。下图是一个继承关系的简单示意图（已略去与本次作业无关的类）。



下面我们将逐一介绍这些子类。

- `pascal.taie.ir.exp.Var`

这个类代表 IR 中的变量。

- `pascal.taie.ir.exp.IntLiteral`

根据 Java 的语言规范 (<https://docs.oracle.com/javase/specs/jls/se11/html/jls-15.html#jls-15.8.1>)，我们在 Tai-e 中把常量称作字面量 (Literals)。每个 `IntLiteral` 类的实例都表示一个程序中的整数字面量。你可以通过调用 `getValue()` 方法来获取它的值。

Tai-e

这个类代表程序中的二元表达式。这个类的各个子类对应了表中的不同种类的二元表达式，并且每个子类中都有一个内部枚举类型 (<https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>) 用于表示该类支持的运算符。例如枚举类型 `ArithmeticExp.Op` 就代表了 `ArithmeticExp` (算术表达式类) 所支持的运算符，也就是 `+` `-` `*` `/` 和 `%`。

需要指出的是，在 Tai-e 中，`BinaryExp` 的两个操作数都是 `Var` 类型的。例如下面的语句

```
1      x = y + 6;                                     java
```

在 Tai-e 中会被转化成如下的 IR:

```
1      %intconst0 = 6;          // %intconst* are temp variables introducedjava
2      x = y + %intconst0; // by Tai-e to hold constant int values
```

这样的设计简化了分析的实现：在获取 `BinaryExp` 的操作数时，你只需要考虑它是变量的这一种可能，而不用担心它是常量或其它可能了。

- `pascal.taie.ir.stmt.DefinitionStmt`

这是 `Stmt` 的一个子类。它表示了程序中所有的赋值语句，(即形如 `x = y` 或 `x = m(...)` 的语句)。这个类很简单。你可以通过阅读源码来决定如何使用它。

- `pascal.taie.analysis.dataflow.analysis.DataflowAnalysis`

这是具体数据流分析算法需要实现的接口。和作业 1 一样，它会被求解器调用。在本次作业中你只需要关注前 5 个 API。这些 API 会被你在后面完成的 worklist 求解器调用。

- `pascal.taie.analysis.dataflow.analysis.constprop.Value`

这个类表示了常量分析中格上的抽象值 (见第 6 讲课件的第 238 页 (<https://cs.nju.edu.cn/tiantan/software-analysis/DFA-FD.pdf#page=238>))。它的代码和注释解释了它的用法。你应该用下列的静态方法获取格上抽象值 (即该类的实例):

- `Value getNAC():` 返回 NAC
- `Value getUndef():` 返回 UNDEF
- `Value makeConstant(int):` 返回给定整数在格上对应的抽象值

- `pascal.taie.analysis.dataflow.analysis.constprop.CPFact`

Tai-e

自 `pascal.taie.analysis.dataflow.fact.MapFact`。这些类的注释都很充分，所以你应该通过阅读源码来决定如何使用其中的 API。

- `pascal.taie.analysis.dataflow.analysis.constprop.ConstantPropagation`

这个类实现了 `DataflowAnalysis`。你需要在其中编写完整的常量传播算法。具体要求见第 2.3 节。

2.3 你的任务 [重点!]

首先，你需要完成 `ConstantPropagation` 的下述 API：

- `CPFact newBoundaryFact(CFG)`
- `CPFact newInitialFact()`
- `void meetInto(CPFact, CPFact)`
- `boolean transferNode(Stmt, CPFact, CPFact)`

你已经在作业 1 中见到过这几个 API，他们是从 `DataflowAnalysis` 中继承下来的，需要注意的是：在实现 `newBoundaryFact()` 的时候，你要小心地处理每个会被分析的方法的参数。具体来说，你要将它们的值初始化为 NAC（请思考：为什么要这么做？）。

提示：

正如第 2.1 节中提到的，本次作业只关注 `int` 类型的常量传播。为了实现这一点，框架代码在 `ConstantPropagation` 类中提供了 `canHoldInt(Var)` 方法来判断一个变量能否储存 `int` 类型的值。你需要利用这个方法来判断一个变量是否在本次作业的分析范围内，并忽略那些不在范围内的变量（例如 `float` 类型的变量）。

此外，你还需要实现下面两个辅助方法：

- `Value meetValue(Value, Value)`

该方法对应着格上的 `meet` 操作（ $\sqcap$ ），见第 6 讲课件的第 238 页 (<https://cs.nju.edu.cn/tiantan/software-analysis/DFA-FD.pdf#page=238>)。你应当在 `meetInto()` 方法中调用它。

Tai-e

这个方法会计算表达式 (exp) 的值 (value)。当然，此处的值是语义上的抽象值。你需要参考第 6 讲课件的第 247 页 (<https://cs.nju.edu.cn/tiantan/software-analysis/DFA-FD.pdf#page=247>) 上的内容来实现它的三种情况。对于其它情况，该方法会像我们在第 2.1 节提到的那样返回 NAC。你应该在 transferNode() 方法中调用它来进行表达式的求值。

提示:

1. 和作业 1 一样，我们对 meetInto() 的设计比较特殊。如果你不记得了，可以再回顾一下作业 1 文档的相关部分。
2. 条件表达式 (如 `a == b`) 的值由 0 (若为 False) 和 1 (若为 True) 来表示。
3. 对于除以 0 的情况 (出现在 / 和 % 中)，我们规定结果为 UNDEF。例如，对于 `x = a / 0`，x 的值将会是 UNDEF。

3 实现 Worklist 求解器

3.1 Tai-e 中你需要了解的类

与迭代求解器类似，你需要清楚 DataflowResult, CFG 和 Solver 的相关用法 (我们已经在作业 1 中介绍过了)。除此之外，你还需要知道：

- `pascal.taie.analysis.dataflow.solver.WorkListSolver`

该类继承了 Solver 类，实现了 worklist 算法。它的实现是不完整的，在本此作业中你需要完成它。

3.2 你的任务 [重点!]

你的第二个任务是完成下述两个 API 的实现：

- `Solver.initializeForward(CFG, DataflowResult)`
- `WorkListSolver.doSolveForward(CFG, DataflowResult)`

考虑到常量传播是一个前向分析，你只需要关注前向分析相关的方法。`initializeForward()` 方法的具体实现可以参考第 6 讲课件的第 258 页 (<https://cs.nju.edu.cn/tiantan/software-analysis/DFA-FD.pdf#page=258>) 上算法的前三行。`doSolveForward()` 则包含了你要实现的算法的主体部分。

Tai-e

1. 讲义中的 worklist 算法通过比较 `old_OUT` 和 `OUT[B]` 来决定后继节点是否应当加入 worklist 中，这个做法比较低效。Tai-e 中 `DataflowAnalysis.transferNode()` 会返回此次 transfer 是否改变了 OUT fact。利用好这一点可以避免多余的判断；
2. 与作业 1 类似，不要忘了在 `Solver.initializeForward()` 中初始化每个语句的 IN 和 OUT。

我们已经给出了框架代码，你的任务是完成代码中被 “TODO - finish me” 注释的部分

4 运行与测试

你可以按我们在 **Tai-e 框架（教学版）配置指南** 中提到的方式来运行分析。在本作业中，Tai-e 对输入类的每个方法进行常量传播分析，并输出分析的结果，也就是每个语句的 OUT fact 所包含的数据流信息（变量的格上对应值）。

```
----- <Assign: void assign()> (constprop) -----
[0@L4] x = 1; null
[1@L5] x = 2; null
[2@L6] x = 3; null
[3@L7] x = 4; null
[4@L8] y = x; null
[5@L8] return; null
```

当你未完成作业的时候，OUT fact 的结果为 `null`，当你完成了所有空缺代码后，分析的输出应当形如：

```
----- <Assign: void assign()> (constprop) -----
[0@L4] x = 1; {x=1}
[1@L5] x = 2; {x=2}
[2@L6] x = 3; {x=3}
[3@L7] x = 4; {x=4}
[4@L8] y = x; {x=4, y=4}
[5@L8] return; {x=4, y=4}
```

Tai-e

我们为本次作业提供了测试驱动类 `pascal.taie.analysis.dataflow.analysis.constprop.CPTest`，你可以按照 **Tai-e 框架（教学版）配置指南** 中描述的方式来测试你的实现是否正确。

5 通用准则

- 在这次作业中，你只需要保证实现的正确性。不必过早考虑效率。
- 禁止**把你的作业包发给其他人参考。
- 禁止**抄袭。自己的工作必须由自己完成。

6 作业提交

你应当提交一个 zip 文件，其中包括你实现好的以下类：

- `ConstantPropagation.java`
- `Solver.java`
- `WorkListSolver.java`

7 评分

你可以参照 **实验作业评测系统使用指南** 来使用我们的作业评测系统对你完成的作业进行评分。

你的分数将取决于你实现的正确性：我们会用你提交的实现来分析 `src/test/resources/` 目录下的测试用例和另外一些未公开的测试用例，然后将得到的结果与标准实现的分析结果进行比较，从而进行评分。

祝君好运!

Last Updated: 2025/12/31 13:13:20

Tai-e

← 作业 1：活跃变量分析和迭代求解器

作业 3：死代码检测 →